Deterministic Tensor Governance via Interface-Led Architecture

A Contract-First Architecture for Governing LLM and Quantum Tensor Providers

Version: v0.1.0
Type: Position Paper / Architecture Note
Author: Takuya Sogawa
ORCID: https://orcid.org/0009-0009-7499-2595
Canonical language: English
DOI: 10.5281/zenodo.20303497

Abstract

This paper proposes a contract-first governance architecture for heterogeneous tensor providers, including Large Language Models (LLMs) and Quantum Processing Units (QPUs).

The paper does not propose a new low-level quantum operating system, quantum compiler, quantum intermediate representation, or hardware control layer. Instead, it defines a higher-level governance discipline for confining partially non-deterministic providers within deterministic interfaces, contracts, invariants, fail-closed boundaries, and auditable execution records.

The central claim of this paper is not that LLM sampling and QPU measurement or noise are physically identical phenomena. Their internal mechanisms, physical meanings, and evaluation methods are fundamentally different. The claim is that, when viewed from a system boundary, both can be treated as architecturally analogous computational resources: each accepts structured input, performs provider-local non-deterministic processing, and returns output that must be verified before it is allowed to affect the surrounding system.

This paper refers to the higher-level design principle for governing such providers through Interface-Led Architecture (ILA) as **Deterministic Tensor Governance**. It refers to the governance-oriented architectural boundary that realizes this principle as **TensorKernel**.

The contribution of this paper is not to reduce LLMs and QPUs to a single physical model. Rather, it provides an initial citable architectural definition for governing non-deterministic tensor providers through software-engineering constructs such as contracts, invariants, replay logs, provider interfaces, policy decision points, and fail-closed execution boundaries.

Keywords

Deterministic Tensor Governance; Interface-Led Architecture; AIKernel; TensorKernel; LLM Governance; Quantum Governance; QPU; Contract-First Architecture; Fail-Closed; ReplayLog; Provider Model; Non-Deterministic Systems; Quantum Middleware; AI Governance

1. Introduction

Modern computing is increasingly concerned not only with the execution of deterministic instruction sequences, but also with the orchestration of probabilistic and partially non-deterministic tensor computation resources. LLMs exhibit non-determinism through probabilistic sampling. QPUs exhibit non-determinism through measurement, noise, device characteristics, and execution conditions.

These systems are not identical internally. An LLM generates token sequences from learned parameters and sampling procedures. A QPU executes quantum circuits through quantum states, gates, measurement, and physical noise, and returns bitstrings, measurement distributions, expectation values, or estimates derived from them.

Nevertheless, from the perspective of an enterprise system or autonomous execution environment, the two share a boundary-level structure. Both receive structured input, pass through provider-local non-determinism, and return output that may affect an external system.

Safely operating such providers requires more than optimization of the computational resource itself. It also requires a deterministic control plane around the provider. This paper extends the ideas developed in the AIKernel project - Interface-Led Architecture, contract-first development, fail-closed governance, and replay logging - into a meta-architectural principle for governing non-deterministic tensor providers, including both LLMs and QPUs.

---

2. Scope and Non-Claims

To avoid overclaiming, this paper first states what it does not propose.

This paper does not propose:

- a new quantum operating system;
- a new quantum compiler;
- a new quantum intermediate representation;
- a new LLM architecture;
- a physical equivalence between LLMs and QPUs;
- a unified physical theory that explains QPU measurement through LLM sampling;
- a replacement for QOS, QNodeOS, Pilot-Quantum, QIR, Qiskit, PennyLane, MLIR, or related systems.

Instead, this paper proposes:

- a higher-level architecture for confining non-deterministic providers within contracts;
- a design principle that separates provider-internal non-determinism from system-level deterministic control;
- an architectural mapping for transferring LLM governance patterns to QPU governance;
- an auditable execution boundary based on fail-closed behavior, replay logs, invariants, and policy decision points;
- a conceptual governance layer that can be placed outside existing quantum operating systems, middleware, intermediate representations, SDKs, and runtimes.

Accordingly, this paper is a position paper about governance boundaries rather than a paper about low-level computation control.

3. Related Work

TensorKernel and Interface-Led Architecture (ILA) are not intended to replace existing quantum operating systems or hybrid middleware. Rather, they respect those systems as providers, runtimes, or intermediate representations, and define a higher-level contract-first governance layer around them.

3.1 Quantum Operating Systems and Middleware

QOS has been proposed as a quantum operating system for quantum resource management, hardware constraints, noise, scheduling, and multiprogramming. It is an important contribution at the lower execution and resource-management layer. In contrast, TensorKernel is positioned as a governance layer outside such systems.

QNodeOS has been proposed as an operating-system architecture for executing applications on quantum network nodes. From the perspective of this paper, QNodeOS is a lower-level execution environment and can be treated as a provider or runtime beneath the governance layer.

Pilot-Quantum is a middleware system for resource, workload, and task management in Quantum-HPC environments. It provides an important basis for execution management in hybrid quantum-classical environments. The governance layer proposed in this paper is placed outside such execution management systems and focuses on execution authorization, provider selection, contract validation, and auditability.

3.2 Quantum Frameworks and Intermediate Representations

Quantum programming environments such as Q#, Qiskit, and PennyLane can be treated as provider implementation foundations or user-facing programming layers.

Quantum Intermediate Representation (QIR) is an intermediate representation between quantum programming languages or frameworks and quantum computing platforms. In this paper, QIR is positioned as a strong candidate for an intermediate representation beneath the TensorKernel provider boundary.

3.3 ML Compiler and Tensor Runtime

Compiler and runtime infrastructures such as MLIR and XLA are important for heterogeneous hardware abstraction, transformation, optimization, and execution efficiency. Their main concern, however, is compilation, optimization, and runtime execution. The concern of this paper is how to govern optimized or provider-generated outputs through higher-level contracts, invariants, policies, and replay logs.

3.4 AIKernel and Interface-Led Architecture

AIKernel treats LLMs not as simple API calls, but as execution targets controlled through context, contracts, providers, replay logs, and governance boundaries. The author's Interface-Led Architecture (ILA) defines a method for building software around interfaces, contracts, and responsibility boundaries, and for confining AI-generated implementation inside deterministic structure.

This paper extends the ILA approach from LLM governance to the governance of non-deterministic tensor providers, including QPUs.

4. Problem Statement

Existing hybrid computation architectures face the following problems.

4.1 Provider-Local Non-Determinism

The non-determinism of LLMs and QPUs should be localized inside providers. However, when contracts, invariants, replay logs, and policy decision points are missing, provider-local non-determinism can leak into the surrounding system.

This leads to several problems:

- the system cannot explain under what conditions an output was accepted;
- it becomes unclear at which boundary the system should stop on failure;
- provider replacement may destabilize the behavior of the whole system;
- verifiability of execution results declines;
- auditability cannot be preserved.

4.2 Absence of Deterministic Control Boundaries

When a system encounters unknown complexity, unacceptable noise, low-confidence output, or a dangerous action candidate, it requires strict architectural contracts that define when execution must stop safely.

Non-determinism cannot always be eliminated. It must instead be localized inside providers and observed, evaluated, limited, and recorded by an external control plane.

The central question of this paper is therefore:

> In a system that uses non-deterministic providers, how can non-determinism be confined within contracts while preserving deterministic governance at the system level?

---

5. Non-Deterministic Tensor Provider Model

This paper abstracts LLMs and QPUs as **Non-Deterministic Tensor Providers**.

Here, the term "tensor" is not limited to numerical tensors in the narrow sense. It refers broadly to structured boundary-level data representations, including high-dimensional inputs and outputs, token sequences, probability distributions, measurement distributions, bitstrings, expectation values, circuit descriptions, and execution metadata.

5.1 LLM Provider

An LLM provider can be represented conceptually as follows:

```
Context / Prompt
   ↓
Probabilistic Sampling
   ↓
Token Sequence / Probability Distribution / Tool Candidate
```

The output of an LLM provider may appear as natural language, structured JSON, a tool candidate, a code fragment, or a reasoning trace. Such output should not be trusted directly. It must be verified by contracts, schemas, policies, and risk evaluation.

5.2 QPU Provider

A QPU provider can be represented conceptually as follows:

```
Quantum Circuit / QIR / Execution Request
   ↓
Quantum Execution / Measurement / Noise
   ↓
Bitstrings / Counts / Expectation Values / Measurement Statistics
```

The output of a QPU provider is often not a single deterministic value. It may be returned as measurement results, distributions, statistical estimates, or error-bearing values. Its meaning must therefore be evaluated through post-processing, error mitigation, confidence criteria, and contract validation.

5.3 Common Boundary Model

LLMs and QPUs are not internally identical. At the architectural boundary, however, they share the following structure:

```
Structured Input
   ↓
Provider-Local Non-Determinism
   ↓
Structured Output
   ↓
Deterministic Contract Validation
   ↓
Governance Decision
```

This common boundary model allows providers with different physical and computational properties to be governed by a unified contract-first model from outside the provider boundary.

---

6. Interface-Led Architecture for Tensor Governance

Interface-Led Architecture (ILA) starts not from implementation or hardware characteristics, but from contracts between system boundaries.

In Tensor Governance, ILA is based on the following principles.

6.1 Contract-First

Before implementing or invoking a provider, the system defines:

- input format;
- output format;
- acceptable uncertainty;
- required metadata;
- failure conditions;
- invariants;
- verification method;
- audit-log requirements.

A contract does not depend on the provider's internal implementation. A provider is replaceable as long as it satisfies the contract.

6.2 Invariant Preservation

An invariant is a condition that must not be violated, regardless of the provider's internal non-determinism.

Examples include:

- invalid output must not be treated as success;
- unverifiable output must not be executed automatically;
- dangerous actions must not be executed without confirmation;
- governance decisions must be recorded in the ReplayLog;
- provider failure must not be hidden as a safe success;
- unknown risk must not be treated as low risk.

6.3 Fail-Closed

Fail-closed is the principle that execution stops at the boundary when provider output fails to satisfy a contract or invariant.

Fail-closed behavior is not merely exception handling. It is the governance boundary that prevents a non-deterministic provider from being directly connected to the rest of the system.

6.4 Provider Replacement Boundary

Providers are placed behind interfaces. This allows LLMs, QPUs, simulators, classical tensor runtimes, and hybrid runtimes to be handled by a common governance layer.

```
ITensorProvider
 ├── LLMProvider
 ├── QPUProvider
 ├── QuantumSimulatorProvider
 ├── ClassicalTensorProvider
 └── HybridRuntimeProvider
```

The goal is not to unify provider internals. The goal is to make provider boundaries observable, verifiable, recordable, and rejectable.

---

7. Deterministic Governance Skeleton

A TensorKernel for safely operating non-deterministic providers is composed of the following governance skeleton:

```
Request
  ↓
Policy Decision Point
  ↓
Contract Selection
  ↓
Provider Routing
  ↓
Provider Execution
  ↓
Result Verification
  ↓
Fail-Closed Gate
  ↓
ReplayLog
  ↓
Approved Output / Rejected Output
```

7.1 Policy Decision Point

The Policy Decision Point (PDP) evaluates the nature, complexity, risk, required confidence level, and available providers before execution.

The PDP decides:

- which provider to route to;
- whether confirmation is required before execution;
- which contract to apply;
- how failures should be handled;
- what evidence must be preserved for the output.

7.2 Contract Registry

The Contract Registry manages provider contracts.

Example:

```
Contract
 ├── Input Schema
 ├── Output Schema
 ├── Invariants
 ├── Risk Policy
 ├── Verification Rule
 ├── Replay Requirement
 └── Fail-Closed Behavior
```

7.3 Provider Adapter

A Provider Adapter maps the provider-specific API, SDK, runtime, or intermediate representation to the common TensorKernel interface.

For QPUs, an adapter may connect to QIR, Qiskit, PennyLane, or a specific quantum runtime. For LLMs, an adapter may connect to an OpenAI-compatible API, a local model runtime, or a tool-calling API.

7.4 Result Verifier

The Result Verifier determines whether provider output satisfies the applicable contract.

For LLMs, it may verify JSON schema validity, tool-call validity, semantic alignment, risk score, and policy compliance.

For QPUs, it may verify shot count, measurement distribution, confidence interval, error threshold, expected observable consistency, and backend metadata.

7.5 ReplayLog

The ReplayLog records all governance decisions and provider inputs and outputs as immutable evidence.

The ReplayLog does not remove non-determinism inside the provider. It does, however, make the control-plane decision path reconstructable.

A replay record may include:

- trace id;
- request metadata;
- selected contract;

- selected provider;
- provider input;
- provider output summary;
- verification result;
- risk score;
- decision;
- fail-closed reason;
- timestamp;
- versioned policy id.

---

8. LLM-to-QPU Governance Transfer

The distinctive contribution of this paper is the transfer of the LLM governance model, specified and validated in AIKernel, to governance over QPU resources.

Here, "transfer" does not imply physical identity. It refers to the transfer of governance patterns at the architectural boundary.

8.1 Analogical Mapping

LLM Governance	QPU Governance
Prompt / Context	Quantum Circuit / QIR / Execution Request
Sampling	Measurement / Device Noise / Shot Execution
Token Sequence	Bitstrings / Counts / Expectation Values
Tool Candidate	Quantum Job / Backend Execution Result
Schema Validation	Measurement Result Validation
Semantic Alignment	Observable / Circuit Intent Consistency
Risk Evaluation	Backend / Noise / Cost / Confidence Evaluation
ReplayLog	Experiment / Execution / Governance Log
Fail-Closed Gate	Reject / Re-run / Require More Shots / Abort

This mapping enables design principles developed for LLM governance to be transferred to QPU governance in an architecturally analogous way.

8.2 Structured Governance Pipeline

The LLM governance pipeline can be represented conceptually as follows:

```
Prompt / Context Structuring
   ↓
Stochastic Generation
   ↓
Deterministic Verification
   ↓
Governance Decision
```

Transferred to QPU governance, it becomes:

```
Circuit / QIR Structuring
   ↓
Probabilistic Measurement
   ↓
Deterministic Post-Processing and Contract Verification
   ↓
Governance Decision
```

This correspondence does not mean that LLMs and QPUs share the same internal mechanism. It means that a common structure exists for governing both from outside as non-deterministic providers.

---

9. Architectural Layering

TensorKernel does not replace existing quantum operating systems, quantum middleware, quantum intermediate representations, LLM runtimes, or ML compilers. It is placed outside them.

```
Application / User Intent
        ↓
TensorKernel Governance Layer
        ↓
Contract / Policy / ReplayLog
        ↓
Provider Adapter
        ↓
QOS / QNodeOS / Pilot-Quantum / QIR / Qiskit / PennyLane / LLM Runtime
        ↓
Hardware / Model / Simulator
```

In this structure, lower layers provide execution capability. TensorKernel governs authorization, rejection, auditing, and contract validation.

---

10. Security and Auditability

In systems that use non-deterministic providers, provider output must be treated as outside the trust boundary.

TensorKernel adopts the following principles:

1. Provider output is untrusted by default.
2. Execution requires explicit governance approval.
3. Unknown risk increases the risk score.
4. Invalid or unverifiable output must not be silently accepted.
5. Dangerous execution requires confirmation or rejection.
6. Provider routing must be policy-controlled.
7. ReplayLog must preserve evidence for governance decisions.
8. Fail-closed behavior must be the default.

These principles prevent LLM or QPU output from directly affecting business systems, execution environments, or physical resources without governance.

---

11. Limitations

This paper has several limitations.

11.1 Conceptual Stage

This paper is a position paper. It does not present a complete TensorKernel implementation or empirical evaluation results. Its claims are therefore limited to architectural principles.

11.2 Analogy Limitation

LLMs and QPUs are fundamentally different in their internal mechanisms. The mapping in this paper is based on architectural analogy at the system boundary, not physical isomorphism.

11.3 Verification Limitation

Contract validation is not universal. For QPUs in particular, validation depends on statistical confidence intervals, shot counts, noise models, and backend metadata. Verification itself may therefore have statistical characteristics.

11.4 Governance Overhead

A PDP, Contract Registry, Result Verifier, and ReplayLog may increase execution cost. Low-risk and low-cost tasks may therefore require lightweight governance profiles, while high-risk execution requires stricter profiles.

11.5 Terminology Risk

The term TensorKernel may be misunderstood as referring to a low-level quantum OS kernel or GPU kernel. In this paper, TensorKernel refers to a contract-first governance layer, not a low-level execution kernel.

---

12. Future Work

Future work includes:

1. defining minimal interfaces such as `ITensorProvider`, `ITensorContract`, and `ITensorReplayLog`;
2. implementing a governance prototype for LLM providers on AIKernel.NET;
3. conducting pseudo-QPU governance experiments using a quantum simulator provider;
4. designing a provider adapter for QIR or Qiskit inputs;
5. evaluating the auditability of provider execution using ReplayLog;
6. modeling initial risk scores and confidence criteria;
7. exploring algebraic extensions such as many-valued logic, p-adic spaces, and probabilistic type systems.

---

13. Conclusion

This paper proposed a higher-level design principle for treating LLMs and QPUs as architecturally analogous Non-Deterministic Tensor Providers while preserving the differences in their internal mechanisms and physical meanings.

The purpose of this paper is not to replace existing quantum operating systems, quantum compilers, quantum middleware, intermediate representations, or LLM runtimes. Rather, it is to respect them as providers, runtimes, or intermediate representations, and to place a contract-first governance layer

around them. This allows a system to tolerate non-determinism while preserving deterministic control, auditability, and fail-closed safety at the architectural level.

Deterministic Tensor Governance does not reduce LLMs and QPUs to a single physical theory. It is an architectural principle for governing non-deterministic computational resources through software-engineering constructs such as contracts, invariants, interfaces, replay logs, and policy decision points.

References

1. Giortamis, Emmanouil, Francisco Romão, Nathaniel Tornow, and Pramod Bhatotia. “QOS: Quantum Operating System.” 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25), USENIX Association, 2025, pp. 429-447.
2. Delle Donne, C., M. Iuliano, B. van der Vecht, G. M. Ferreira, H. Jirovská, T. J. W. van der Steenhoven, A. Dahlberg, et al. “An Operating System for Executing Applications on Quantum Network Nodes.” Nature, vol. 639, no. 8054, 2025, pp. 321-328. DOI: 10.1038/s41586-025-08704-w.
3. Mantha, Pradeep, Florian J. Kiwit, Nishant Saurabh, Shantenu Jha, and Andre Luckow. “Pilot-Quantum: A Quantum-HPC Middleware for Resource, Workload and Task Management.” arXiv:2412.18519, 2024. DOI: 10.48550/arXiv.2412.18519.
4. Microsoft. “Q# Quantum Programming Language.” Microsoft Learn.
5. IBM. “Qiskit SDK.” IBM Quantum Documentation.
6. Bergholm, Ville, Josh Izaac, Maria Schuld, Christian Gogolin, M. Sohaib Alam, Shahnawaz Ahmed, Juan Miguel Arrazola, et al. “PennyLane: Automatic Differentiation of Hybrid Quantum-Classical Computations.” arXiv:1811.04968, 2018.
7. Microsoft. “Quantum Intermediate Representation.” Microsoft Learn.
8. Lattner, Chris, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, et al. “MLIR: Scaling Compiler Infrastructure for Domain Specific Computation.” 2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), 2021.
9. Sogawa, Takuya. “AIKernel Trajectory Governance Model: A Kernel-Level Framework for Convergent Decision Control over Stochastic Language Model Inference.” AIKernel.NET Phase-1 Specification, 2026.
10. Sogawa, Takuya. “Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model.” Zenodo, 2026. DOI: 10.5281/zenodo.20290614.